Ejercicio 1

Quiero implementar en Java una lista enlazada simple de numeros enteros desde cero sin utilizar ArrayList ni LinkedList ni ninguna estructura de la libreria de Java. Necesito dos clases: una clase Nodo que contenga el dato entero y una referencia llamada siguiente que apunte al proximo nodo (o null si es el final), y la clase ListaEnlazada que administre la lista. La referencia head o cabeza es el unico punto de entrada que conoce la lista para no perder los datos, y que cada vez que se inserta un nodo se incrementa una variable size y al borrar se decrementa. La lista debe tener los metodos: insertarAlInicio(int dato), insertarAlFinal(int dato), imprimir(), estaVacia() y getSize(). Para insertar al inicio se debe conectar primero el nodo nuevo a head y luego mover head para no perder la lista.

Ejercicio 2

A partir de la lista enlazada simple de enteros, necesito implementar en Java un metodo boolean buscar(int dato) que devuelva true si el numero existe dentro de la lista o false si no esta. No se puede acceder directamente por un indice como hariamos en un arreglo comun (por ejemplo vector[i]) y porque si o si estamos obligados a hacer un recorrido secuencial nodo a nodo desde head usando un nodo auxiliar caminante hasta encontrar la coincidencia o toparnos con null. El metodo debe arrancar desde actual = head e ir avanzando con actual.getSiguiente(). El codigo debe contemplar el caso de una lista vacia devolviendo false.

Ejercicio 3

Quiero implementar en Java una lista enlazada simple con el metodo int obtener(int posicion) que retorne el valor numerico guardado en una posicion determinada de la lista. Antes de recorrer se deben validar las posiciones invalidas: si posicion < 0 o si posicion >= size, en cuyo caso debe lanzar una excepcion para no romper el programa. Aunque le pasemos una posicion numerica, la lista enlazada no tiene acceso como un vector (que seria, sino que internamente tiene que hacer un bucle for o while avanzando secuencialmente posicion veces desde head para llegar al nodo deseado.

Ejercicio 4

Necesito implementar en Java el metodo void insertarEnPosicion(int dato, int posicion) sobre la lista enlazada simple. El metodo debe permitir insertar en tres casos: al inicio, en el medio de la lista o al final de todo (si posicion es igual a size). Debe validar rangos invalidos (posicion < 0 o posicion > size). Primero se debe conectar nuevo.setSiguiente(actual.getSiguiente()) para enlazar el nuevo con el resto de la lista, y recien despues hacer actual.setSiguiente(nuevo). Si actualizamos primero actual hacia nuevo, perderiamos la referencia a todos los nodos siguientes dejandolos irrecuperables en memoria.

Ejercicio 5

Quiero implementar en Java un metodo boolean eliminar(int dato) en la lista enlazada simple de enteros, el cual debe buscar y eliminar unicamente la primera aparicion del dato especificado devolviendo true si lo borro o false si no existia. El algoritmo debe contemplar todos los casos limite: si la lista esta vacia, si hay que eliminar justo el primer nodo (head), si el dato esta en el medio o si esta en el ultimo nodo de la cadena. En Java no existe un comando para destruir fisicamente el nodo de la memoria, sino que simplemente se desvinculan las referencias puenteando el nodo a borrar, y luego el Garbage Collector se encarga de reclamar esa memoria automaticamente al ver que nadie mas apunta a el. El codigo debe decrementar el size si borro y estar bien comentado.

Ejercicio 6

Se requiere implementar en Java el metodo void eliminarEnPosicion(int posicion) para la lista enlazada. Lo primero que debe hacer el metodo es validar que la posicion sea valida (posicion >= 0 y posicion < size), lanzando una excepcion si esta fuera de rango. Si la posicion es 0, debe eliminar el primer nodo actualizando head = head.getSiguiente(). Para cualquier otra posicion intermedia o final, se debe explicar como avanzar con un bucle hasta ubicarse exactamente en el nodo anterior al que queremos borrar, y luego puentearlo haciendo que el anterior apunte al nodo siguiente del eliminado (anterior.setSiguiente(anterior.getSiguiente().getSiguiente())). El metodo tiene que restar 1 al atributo size obligatoriamente.

Ejercicio 7

Quiero implementar en Java el metodo void modificar(int posicion, int nuevoDato) para cambiar el valor numerico guardado en una posicion dada de la lista. Primero debe verificar que la posicion exista dentro de los limites de la lista (posicion >= 0 && posicion < size) antes de ponerse a recorrer. La diferencia conceptual entre modificar el dato de un nodo y modificar una referencia es que no se tocan los punteros siguiente ni se reacomodan enlaces en la memoria, solo se llega al nodo caminando hasta la posicion y se actualiza su variable de carga util con actual.setDato(nuevoDato) manteniendo intacta toda la estructura de la lista.

Ejercicio 8

Necesito implementar en Java el metodo int contarOcurrencias(int dato) en la lista enlazada simple. Este metodo tiene que recorrer completamente la lista desde head hasta el final (null) y llevar un contador entero que se sume en 1 cada vez que encuentre un nodo cuyo dato sea igual al valor buscado. A diferencia del metodo buscar() aca no se debe cortar el recorrido con un break o un return cuando encontramos la primera coincidencia, ya que en una lista desordenada pueden existir elementos duplicados mas adelante y para dar la cuenta exacta es obligatorio visitar todos los nodos hasta el final. El codigo debe estar comentado y probarse con el ejemplo: 10 -> 20 -> 10 -> 30 -> 10 -> null buscando el numero 10 (que debe devolver 3) y con un numero que no este (que devuelva 0).

Ejercicio 9

Quiero implementar en Java el metodo void invertir() sobre la lista enlazada simple de enteros para dar vuelta todos sus elementos sin crear otra lista auxiliar. Por ejemplo si la lista tiene 10 -> 20 -> 30 -> 40 -> null, al invertirla debe quedar 40 -> 30 -> 20 -> 10 -> null. Se necesitan tres punteros que son anterior (inicializado en null), actual (arranca en head) y siguiente (para guardar temporalmente el resto antes de desconectar). El orden de actualizacion es importante: primero se guarda siguiente = actual.getSiguiente(), despues se invierte la flecha actual.setSiguiente(anterior), luego se avanza anterior = actual y finalmente actual = siguiente. Al salir del bucle se reasigna head = anterior.

Ejercicio 10

Necesito transformar la lista enlazada simple que hicimos con enteros en una version generica completa utilizando tipos genericos de Java: ListaEnlazada<T> y Nodo<T>. La lista debe permitir guardar cualquier tipo de objeto, como por ejemplo ListaEnlazada<Integer>, ListaEnlazada<String> o ListaEnlazada<Alumno>. Cambian las firmas de los metodos reemplazando int por el tipo parametrizado T, y la comparacion de datos ya no puede hacerse con el operador primitivo == sino que es obligatorio utilizar .equals(). La logica de punteros, enlaces, inserciones y eliminaciones no cambia en absoluto, ya que los algoritmos de la lista son independientes del tipo de dato almacenado en el nodo.